LAB 05

Suy luận kết hợp ARM + FPGA trên bo mạch PYNQ-Z2

Slide 06 | Triển khai và benchmark Tiny CNN V5 INT8

Nội dung của bài học

– Hai cách thực thi Tiny CNN V5: **ARM thuần** và **ARM + FPGA**.

– Chuẩn bị tensor INT8, nạp Overlay và thiết lập AXI DMA.

– Đọc mã Notebook: suy luận ARM, suy luận kết hợp và kiểm chứng.

– Đo 30 lần, phân tích thời gian và hiểu giới hạn kết quả.

Mục tiêu của Lab 05

– Chạy mô hình Tiny CNN V5 INT8 thực tế trên **ARM Cortex-A9**.

– Chuyển **Conv1 + ReLU + MaxPool1** sang IP phần cứng của FPGA.

– Ghép tensor từ FPGA với **TFLite tail** chạy trên ARM.

– So sánh đầu ra và độ trễ của hai phương án trên **cùng một ảnh MRI**.

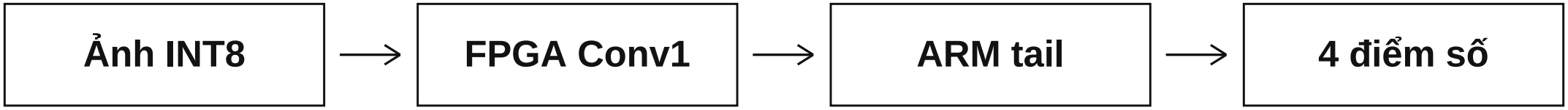
Hai phương án suy luận

Dùng cùng ảnh INT8 và cùng tham số mô hình.

ARM thuần



ARM + FPGA



Vì sao chỉ tăng tốc Conv1?

Phân chia phần mềm và phần cứng ở đầu ra của MaxPool1.

– Conv1 là khối tích chập đầu tiên; FPGA thực hiện chung với **ReLU** và **MaxPool1**.

– Đầu ra sau Pool1 có kích thước **47 × 47 × 8** và có thể đưa trực tiếp vào phần mạng còn lại.

– Phần **ARM tail** thực hiện các lớp Conv2–Conv4 và phần phân loại.

– Thiết kế này tránh tính lại Conv1 trên ARM ở phương án kết hợp.

Dữ liệu đi qua hệ thống

Hình dạng dữ liệu phải khớp chính xác tại ranh giới ARM – FPGA.

Giai đoạn	Kích thước tensor	Số byte INT8
Ảnh đầu vào	96 × 96 × 1	9.216
Đầu ra Conv1	94 × 94 × 8	70.688
Sau ReLU + MaxPool1	47 × 47 × 8	17.672
Đầu ra phân loại	1 × 4	4

Phần mềm chỉ nhận trực tiếp tensor sau Pool1; FPGA không cần chuyển toàn bộ feature map Conv1 về ARM.

Tiền xử lý ảnh MRI trên PYNQ-Z2

– Đọc ảnh **brisc2025_test_00023_gl_ax_t1.jpg**.

– Đưa về ảnh xám một kênh (**grayscale**).

– Resize về **96 × 96** bằng nội suy song tuyến tính (**bilinear**).

– Trừ 128 khỏi giá trị pixel rồi ép sang **np.int8**.

Cả ARM thuần và ARM + FPGA cùng sử dụng một tensor x để bảo đảm so sánh công bằng.

Code: tạo tensor INT8 96 × 96

```
IMAGE = "brisc2025_test_00023_gl_ax_t1.jpg"
img = Image.open(IMAGE).convert("L")
img = img.resize((96, 96), Image.BILINEAR)
x = (np.asarray(img, dtype=np.int16) - 128)
x = x.astype(np.int8).reshape(1, 96, 96, 1)
```

– Dùng **int16** khi trừ 128 để tránh tràn số trước khi ép về INT8.

– Tensor x có hình dạng **(1, 96, 96, 1)**.

Những tệp cần có trên board

Tệp	Chức năng
conv1_accel.bit	Bitstream cấu hình FPGA
conv1_accel.hwh	Mô tả metadata phần cứng của Overlay
tiny_cnn_v5_int8.tflite	Mô hình INT8 đầy đủ, chạy trên ARM
tiny_cnn_v5_tail_int8.tflite	Phần mạng sau MaxPool1, chạy trên ARM
Ảnh MRI thử nghiệm	Đầu vào dùng cho bài benchmark

Overlay, IP Conv1 và AXI DMA là gì?

– **Overlay:** cấu hình phần cứng PL bằng bitstream và mô tả HWH.

– **conv1_hls_0:** IP được tạo bởi Vitis HLS, tính Conv1 + ReLU + Pool1.

– **axi_dma_0:** khối truyền dữ liệu giữa DDR và luồng AXI của IP mà không phải gửi từng pixel bằng lệnh CPU.

– **TFLite Runtime:** Interpreter dùng chạy CNN đầy đủ hoặc phần tail trên ARM.

Code: nạp phần cứng và hai mô hình

```
ol = Overlay("conv1_accel.bit")
dma = ol.axi_dma_0
conv1 = ol.conv1_hls_0
full = Interpreter(
    model_path="tiny_cnn_v5_int8.tflite",
    num_threads=1
)
tail = Interpreter(
    model_path="tiny_cnn_v5_tail_int8.tflite",
    num_threads=1
)
full.allocate_tensors()
tail.allocate_tensors()
```

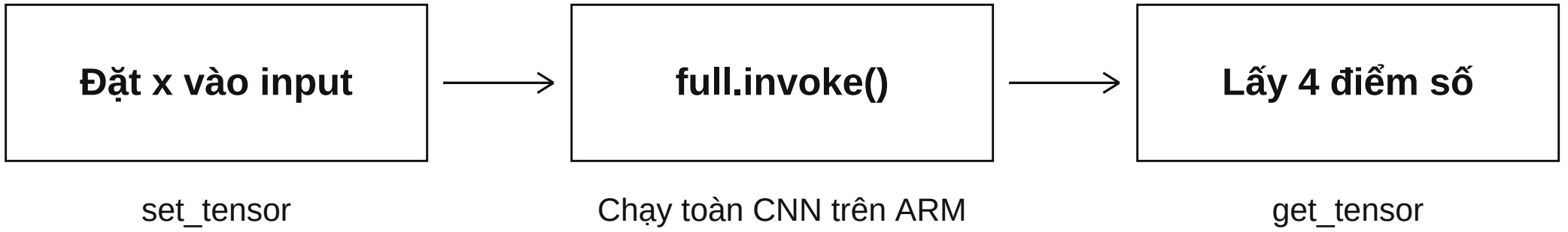
Code: truy cập các tensor và bộ đệm

```
fi = full.get_input_details()[0]["index"]
fo = full.get_output_details()[0]["index"]
ti = tail.get_input_details()[0]["index"]
to = tail.get_output_details()[0]["index"]
ibuf = allocate(shape=(9216,), dtype=np.int8)
obuf = allocate(shape=(47, 47, 8), dtype=np.int8)
```

– **ibuf**: bộ đệm dữ liệu gửi tới FPGA; **obuf**: vùng nhận kết quả sau MaxPool1.

– Bộ đệm được cấp phát qua PYNQ để DMA có thể truy cập.

Suy luận ARM thuần diễn ra thế nào?



– Thời gian bao gồm thiết lập tensor, gọi Interpreter và lấy đầu ra.

– Mô hình được nạp sẵn; không đo thời gian khởi tạo TFLite.

Code: hàm chạy ARM thuần

```
def run_arm():
    t0 = time.perf_counter()
    full.set_tensor(fi, x)
    full.invoke()
    result = full.get_tensor(fo)
    ms = (time.perf_counter() - t0) * 1000
    return result, ms
```

time.perf_counter() dùng đo khoảng thời gian thực thi có độ phân giải cao; kết quả đổi sang mili giây.

Suy luận ARM + FPGA: luồng thực thi



Trình tự điều khiển

– DMA gửi 9.216 byte ảnh và nhận 17.672 byte kết quả.

– Sau khi DMA hoàn tất: invalidate cache, reshape tensor và gọi Interpreter tail.

Code: bắt đầu phép truyền DMA

```
def run_hybrid():
    t0 = time.perf_counter()
    ibuf[:] = x.reshape(-1)
    ibuf.flush()
    dma.recvchannel.transfer(obuf)
    conv1.write(0x00, 1)
    dma.sendchannel.transfer(ibuf)
```

– **flush():** đồng bộ dữ liệu input cho DMA đọc.

– **recvchannel:** chuẩn bị luồng nhận trước khi gửi.

Tại sao cần kiểm tra trạng thái DMA?

– Chương trình phải biết cả luồng gửi **MM2S** và luồng nhận **S2MM** đã hoàn tất.

– Nếu một kênh báo lỗi, cần dừng thay vì lấy tensor không hợp lệ.

– Notebook sử dụng **polling** trạng thái và giới hạn chờ **3 giây**.

– Không dùng lệnh sleep cố định vì có thể vừa chậm vừa không đảm bảo hoàn tất.

Code: kiểm tra lỗi và hoàn tất DMA

```
deadline = time.monotonic() + 3.0
while True:
    tx = dma.mmio.read(0x04)
    rx = dma.mmio.read(0x34)
    if (tx | rx) & 0x70:
        raise RuntimeError("DMA error")
    if (tx & 2) and (rx & 2):
        break
    if time.monotonic() >= deadline:
        raise TimeoutError("DMA timeout")
```

Mã kiểm tra hai kênh DMA: đọc trạng thái, báo lỗi và kiểm soát timeout.

Code: nhận tensor và chạy ARM tail

```
obuf.invalidate()
t_fpga = time.perf_counter()
pool1 = obuf.reshape(1, 47, 47, 8)
tail.set_tensor(ti, pool1)
tail.invoke()
result = tail.get_tensor(to)
t_end = time.perf_counter()
```

– **invalidate():** làm mới cache để CPU đọc dữ liệu DMA vừa ghi.

– **reshape():** tạo đúng tensor 47 × 47 × 8 cho TFLite tail.

Vì sao phải kiểm chứng đầu ra?

Tăng tốc có ý nghĩa khi kết quả vẫn tương đương phương án tham chiếu.

```
a, _ = run_arm()

b, _, _, _ = run_hybrid()

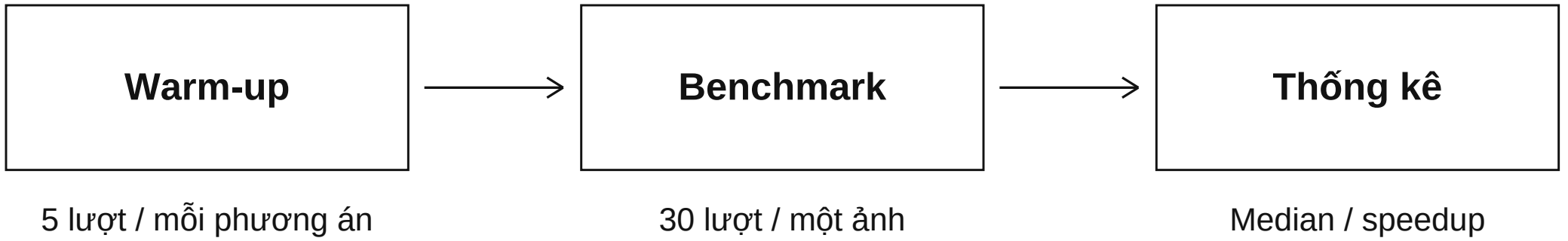
assert np.array_equal(a, b), "Output mismatch"
```

– **np.array_equal** kiểm tra toàn bộ giá trị đầu ra INT8 theo đúng từng phần tử.

– Nếu khác bất kỳ phần tử nào, phép so sánh không đạt.

– Lab 05 xác nhận **đầu ra khớp ở toàn bộ 30 lượt benchmark**.

Thiết kế phép đo công bằng



– Warm-up giúp giảm ảnh hưởng của một số công việc khởi tạo khi chạy lần đầu.

– Luân phiên thứ tự chạy ARM và Hybrid giữa các lượt để hạn chế thiên lệch thứ tự.

Code: 5 lượt warm-up, 30 lượt đo

```
for _ in range(WARMUP):
    a, _ = run_arm()
    b, _, _, _ = run_hybrid()
    assert np.array_equal(a, b)
for i in range(RUNS):
    if i % 2 == 0:
        a, arm_ms = run_arm()
        b, fpga_ms, tail_ms, hybrid_ms = run_hybrid()
    else:
        b, fpga_ms, tail_ms, hybrid_ms = run_hybrid()
        a, arm_ms = run_arm()
```

Vòng lặp còn lưu thời gian từng lượt vào danh sách và kiểm tra đầu ra trước khi thống kê.

Đọc số liệu bằng trung vị (Median)

– **Median:** giá trị đứng giữa khi các mẫu thời gian đã được sắp xếp.

– Kém nhạy với một vài lượt quá chậm hoặc quá nhanh hơn so với trung bình.

– Notebook lấy trung vị riêng cho **ARM**, **FPGA + DMA**, **ARM tail** và **Hybrid**.

– Hệ số tăng tốc được tính từ trung vị ARM chia cho trung vị Hybrid.

Hình minh chứng kết quả thực tế

```
Warming up...
Benchmarking 30 runs...

=== MRI CLASSIFICATION ===
Image: brisc2025_test_00023_gl_ax_t1.j
Predicted class index: 0
Scores: [0.9961 0.     0.     0.    ]
ARM output: [ 127 -128 -128 -128]
Hybrid output: [ 127 -128 -128 -128]
All outputs match: True

=== PERFORMANCE (MEDIAN) ===
Measured runs: 30
ARM only       : 23.626 ms
FPGA + DMA     : 1.507 ms
ARM tail       : 16.554 ms
ARM + FPGA     : 18.055 ms
Speedup        : 1.309x

PASS: Hybrid CNN verified
```

Ảnh output trực tiếp của Notebook Lab_05_Conv1_Test.ipynb trong báo cáo Lab 05.

Kết quả benchmark: 30 lượt chạy

Chỉ tiêu	Giá trị
ARM thuần (median)	23,626 ms
FPGA + DMA (median)	1,507 ms
ARM tail (median)	16,554 ms
ARM + FPGA (median)	18,055 ms
Speedup	1,309×
Đầu ra hai phương án	Khớp 30/30 lượt

Tính mức tăng tốc

So sánh trung vị ARM thuần với trung vị ARM + FPGA.

Speedup = T_ARM / T_Hybrid

= 23,626 / 18,055 ≈ 1,309×

Giảm thời gian: 23,626 − 18,055 = 5,571 ms/ảnh

Con số 1,309× chỉ áp dụng cho phép đo Lab 05 trên một ảnh MRI đã chuẩn bị sẵn.

Vì sao 1,507 + 16,554 khác 18,055?

Không cộng các trung vị thành phần để suy ra trung vị của tổng.

– **FPGA + DMA** và **ARM tail** được đo trên từng lượt chạy.

– **Hybrid** là thời gian thực tế của cả chuỗi từ đầu đến cuối.

– Mỗi cột thời gian được lấy trung vị riêng sau 30 lượt.

– Vì thế **1,507 + 16,554 = 18,061 ms** có thể khác **18,055 ms**.

Đầu ra phân loại có ý nghĩa gì?

Thông tin	Giá trị trên ảnh đã thử
Predicted class index	0 (glioma theo thứ tự nhãn)
ARM output INT8	[127, −128, −128, −128]
Hybrid output INT8	[127, −128, −128, −128]
Điểm số giải lượng tử của lớp 0	Xấp xỉ 0,9961

Một ảnh được dự đoán với điểm số 0,9961 không có nghĩa mô hình đạt 99,61% Accuracy trên tập Test.

Phép đo bao gồm và không bao gồm gì?

Được tính vào độ trễ	Không tính vào độ trễ
Thiết lập tensor đầu vào	Đọc file ảnh MRI
TFLite invoke / ARM tail	Chuyển grayscale và resize
Truyền DMA và xử lý FPGA	Nạp bitstream / tạo Overlay
Đọc tensor đầu ra	Khởi tạo Interpreter và cấp phát bộ đệm

Kết luận kỹ thuật của Lab 05

– Hệ thống kết hợp đã chạy được **FPGA Conv1 + ReLU + Pool1** và **ARM tail**.

– Đầu ra INT8 của hai phương án bằng nhau trong phép thử 30 lượt.

– Thời gian trung vị Hybrid là **18,055 ms**, nhanh hơn ARM thuần **1,309×**.

– Kết quả này mới đo trên **một ảnh MRI**, chưa đủ đại diện cho toàn bộ tập kiểm thử.

Lab 05 dẫn tới Lab 06 như thế nào?

– **Lab 05:** xác nhận hệ thống ARM + FPGA hoạt động đúng trên một ảnh; benchmark 30 lượt.

– **Lab 06:** mở rộng đánh giá trên **995 ảnh Test** để đo Accuracy, F1 và thời gian suy luận.

– Không dùng kết quả điểm số của một ảnh để suy luận hiệu quả phân loại cho toàn tập.

– Đây là lý do cần tách **kiểm thử chức năng** và **đánh giá thống kê**.

Câu hỏi ôn tập

– Vì sao **tail.tflite** không được chứa lại Conv1 + ReLU + Pool1?

– Sự khác nhau giữa **flush()** và **invalidate()** là gì?

– Vì sao cần đợi **cả hai kênh DMA** hoàn tất?

– Tại sao phải warm-up, chạy 30 lượt và luân phiên thứ tự?

– 1,309× thể hiện điều gì, và điều gì chưa thể kết luận từ Lab 05?